

Tarea 2: Programación Intermedia (6%)

Marlon Brenes*

El propósito de esta tarea es practicar los conceptos de C++ y OOP en C++. La tarea involucra entregar código fuente de C++ con su respectivo reporte detallando el análisis de sus resultados y la explicación de su implementación. Usted debe entregar los siguientes archivos para ser evaluados:

- `matrix.cpp`, `matrix.hpp` y `main.cpp`
- `rk4.cpp`
- `reporte3.pdf/reporte3.doc/reporte3.docx/reporte3.ipynb` (dependiendo de cuál formato escoja para su reporte)

Note que para la Parte II se debe entregar un gráfico. Explique los resultados del gráfico en su reporte.

PARTE I

Su tarea es construir una clase que puede ser utilizada para trabajar con matrices. La clase maneja un recurso de memoria dinámico asignado en el heap, cuyos datos contenidos corresponden a la matriz organizada en un arreglo unidimensional (memoria contigua) como se ha discutido en clase.

1. Genere tres archivos:

- `matrix.hpp`: este archivo contiene la declaración de la clase
- `matrix.cpp`: este archivo contiene la implementación de la clase
- `main.cpp`: este archivo contiene el `main()` con el cual pondremos en uso la funcionalidad de la clase

2. Declare una clase llamada `Matrix` que contiene tres atributos: `int n_cols` para el número de columnas, `int n_rows` para el número de filas y `double *data` que contiene los elementos de la matriz. Decida si los miembros deben ser públicos o privados. Recuerde que si decide que el tamaño de la matriz es público, se pueden llegar a estados inconsistentes del objeto y esto corresponde a un error en la implementación del paradigma. Si los miembros son privados, debe implementar métodos `get_rows()` y `get_cols()` para acceder estos miembros sin ser modificados [0.5%]

3. Decida que hacer con el constructor por defecto y escriba su implementación para no generar un estado inconsistente de los objetos de tipo `Matrix` [0.25%]

4. Genere un constructor personalizado que asigna la memoria de la matriz con el número de filas y de columnas como argumento [0.25%]

5. Genere un constructor copia apropiado (deep copy) [0.25%]

6. Genere un operador de asignación apropiado (dos matrices son iguales si y solo si sus elementos y dimensiones son iguales) [0.25%]

7. Genere un destructor apropiado (note que la clase maneja recursos dinámicos de memoria) [0.25%]

8. Escriba un método para imprimir la matrix en `stdout` [0.25%]

9. Sobrecarga de operadores: Sobrecargue los siguientes operadores y explique porqué en general esto es una buena práctica en su reporte [0.75%]:

- Adición +
- Sustracción -
- Multiplicación *

* marlon.brenes@utoronto.ca

10. En todos los métodos que lo requieran, utilice manejo de excepciones para cerciorar que `n_rows` y `n_cols` satisfacen las condiciones necesarias para la operación requerida [0.25%]
11. Escriba una prueba de cada método y operador en el main. Las operaciones deben dar resultados correctos. La inicialización (i.e., los valores que la matrix contiene) puede hacerse en el main. Realice las pruebas con matrices rectangulares (no cuadradas). [0.5%]

PARTE II

Utilice el método RK4 para evaluar la solución a la ecuación

$$\frac{dy}{dx} = 2(1 - y) - e^{-4x}, \quad (1)$$

con la condición inicial $y(0) = 1$. Note que el lado derecho de la ecuación depende de la variable independiente y la ecuación se refiere a variables escalares. Escriba la solución en su reporte y grafique el resultado de la integración numérica en el rango $x \in [0, 2]$. Pruebe distintos valores de h para comparar su solución. [2.5%].